

RailTime Videoanrufe — Server-Einrichtung (LiveKit + coturn)

Dokumentst and	28.07.2026
Gilt für	Media-VM (LiveKit + Caddy + optional coturn), Phasen 1–3 und 8 des Integrationsplans
Gegenstück	Der App-seitige Code (Phasen 4–7) ist bereits im Repository umgesetzt — siehe LIVEKIT_INTEGRATION_PLAN.md

Diese Anleitung ist zum **Abarbeiten von Hand** gedacht: alle Dateien sind kopierfertig, alle Befehle direkt ausführbar. Platzhalter stehen in spitzen Klammern (<...>).

Inhaltsverzeichnis

0. Überblick und Voraussetzungen	2
1. Phase 1 — Grundinfrastruktur prüfen (Plesk-Server)	2
2. Media-VM vorbereiten	2
3. Phase 3 — LiveKit + Caddy einrichten	3
3.1 API-Schlüssel erzeugen	3
3.2 /opt/railtime-media/livekit.yaml	3
3.3 /opt/railtime-media/Caddyfile	3
3.4 /opt/railtime-media/docker-compose.yml	4
3.5 systemd-Unit /etc/systemd/system/railtime-media.service	4
3.6 App anbinden und prüfen (Plesk-Server)	4
4. Phase 2 — coturn als Härtingsstufe (optional, empfohlen für strenge Firmennetze)	5
4.1 Zertifikat für coturn	5
4.2 /opt/railtime-media/turnserver.conf	5
4.3 Starten und in der App umschalten	5
5. Phase 8 — Test und Absicherung	6
5.1 Funktionsprüfung	6
5.2 Netz-Matrix	6
5.3 Geräte	6
5.4 Last und Chaos	6
5.5 Betrieb	6
Anhang — Plan B: alles auf dem Plesk-Server	7

0. Überblick und Voraussetzungen

Architektur: Der Plesk-Server (app.rail-time.de) behält Laravel, Reverb (nur Signalisierung) und die Queue. Sämtliche **Medienströme** laufen über eine **eigene kleine Media-VM** mit LiveKit. Der Plesk-Host und das Repository bleiben Docker-frei.

Benötigt wird:

- Eine VM mit ~2 vCPU / 4 GB RAM (z. B. Hetzner CX32), Ubuntu 22.04/24.04, öffentliche IPv4.
- Zwei DNS-A-Records auf diese VM: `livekit.rail-time.de` und `turn.rail-time.de`.
- Root-Zugang per SSH.

Portmatrix (Firewall der Media-VM, eingehend):

Port	Protokoll	Zweck
22	tcp	SSH (Adminzugang, ggf. einschränken)
80	tcp	Let's-Encrypt-Challenge (Caddy)
443	tcp	TLS → LiveKit WebSocket (7880)
7881	tcp	ICE/TCP-Fallback
50000–60000	udp	Medienströme (WebRTC)
3478	udp+tcp	TURN
5349	tcp	TURN über TLS
49160–49960	udp	coturn-Relay (nur bei Variante coturn)

Fallback-Leiter für strenge Firmennetze (handeln die Clients automatisch aus): UDP 50000–60000 → TCP 7881 → TURN 3478 → **TURN-TLS 5349/443** (funktioniert fast überall, da von HTTPS nicht unterscheidbar).

1. Phase 1 — Grundinfrastruktur prüfen (Plesk-Server)

Es wird nichts gebaut, nur verifiziert:

- ☐ Reverb läuft und ist von extern über WSS erreichbar (bestehender Plesk-Nginx-Proxy auf Port 8080, Supervisor-Programm `railtime-reverb`).
- ☐ Der Laravel-Queue-Worker läuft (`QUEUE_CONNECTION=database`) — er trägt die Ring-Timeouts der Anrufe.
- ☐ Festlegung bestätigt: **Reverb transportiert nie Medien**, der Plesk-Nginx proxied nie Medienströme.

2. Media-VM vorbereiten

```
ssh root@<media-vm-ip>

# System + Docker Engine
apt-get update && apt-get -y upgrade
curl -fsSL https://get.docker.com | sh

# Firewall (ufw)
apt-get -y install ufw
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
ufw allow 7881/tcp
ufw allow 50000:60000/udp
ufw allow 3478/udp
ufw allow 3478/tcp
ufw allow 5349/tcp
ufw allow 49160:49960/udp
ufw enable

mkdir -p /opt/railtime-media && cd /opt/railtime-media
```

3. Phase 3 — LiveKit + Caddy einrichten

3.1 API-Schlüssel erzeugen

Auf dem **Plesk-Server** im Projektverzeichnis (der Befehl schreibt die Werte zugleich in die `Laravel-.env`):

```
php artisan railtime:livekit-keys --host=livekit.rail-time.de
```

Die Ausgabe enthält den fertigen `keys:-Block` für die `livekit.yaml` unten. Alternativ auf der Media-VM: `docker run --rm livekit/livekit-server:v1.8 generate-keys`.

3.2 `/opt/railtime-media/livekit.yaml`

```
port: 7880
bind_addresses: ["0.0.0.0"]
rtc:
  udp_port_range_start: 50000
  udp_port_range_end: 60000
  tcp_port: 7881
  use_external_ip: true
keys:
  # Aus `php artisan railtime:livekit-keys` uebernehmen:
  <LIVEKIT_API_KEY>: "<LIVEKIT_API_SECRET>"
webhook:
  api_key: <LIVEKIT_API_KEY>
  urls:
    - https://app.rail-time.de/webhooks/livekit
turn:
  enabled: true
  domain: turn.rail-time.de
  tls_port: 5349
  udp_port: 3478
  external_tls: true      # TLS terminiert Caddy (siehe Caddyfile)
logging:
  level: info
```

3.3 `/opt/railtime-media/Caddyfile`

```
livekit.rail-time.de {
    reverse_proxy 127.0.0.1:7880
}

turn.rail-time.de:5349 {
    reverse_proxy 127.0.0.1:3478
}
```

> Caddy besorgt und erneuert die Let's-Encrypt-Zertifikate automatisch (Port 80 muss dafür offen bleiben).

3.4 /opt/railtime-media/docker-compose.yml

```
services:
  livekit:
    image: livekit/livekit-server:v1.8
    network_mode: host          # zwingend: 10.000 UDP-Ports sind per Mapping unbrauchbar
    volumes:
      - ./livekit.yaml:/etc/livekit.yaml:ro
    command: --config /etc/livekit.yaml
    restart: unless-stopped

  caddy:
    image: caddy:2
    network_mode: host
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy-data:/data
    restart: unless-stopped

  coturn:
    # Haertungsstufe, Start nur bei Bedarf (Abschnitt 4)
    image: coturn/coturn:latest
    network_mode: host
    volumes:
      - ./turnserver.conf:/etc/coturn/turnserver.conf:ro
      - /etc/letsencrypt:/etc/letsencrypt:ro
    profiles: ["coturn"]
    restart: unless-stopped

volumes:
  caddy-data:
```

3.5 systemd-Unit /etc/systemd/system/railtime-media.service

```
[Unit]
Description=RailTime Media-Stack (LiveKit + Caddy [+ coturn])
After=docker.service network-online.target
Requires=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/railtime-media
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down

[Install]
WantedBy=multi-user.target
```

```
systemctl daemon-reload
systemctl enable --now railtime-media.service
docker compose ps          # livekit + caddy muessen "running" sein
```

3.6 App anbinden und prüfen (Plesk-Server)

Die Laravel-.env enthält nach railtime:livekit-keys bereits:

```
LIVEKIT_URL=https://livekit.rail-time.de
LIVEKIT_WS_URL=wss://livekit.rail-time.de
LIVEKIT_API_KEY=<...>
LIVEKIT_API_SECRET=<...>
LIVEKIT_TURN_MODE=embedded
```

Dann:

```
php artisan config:clear
php artisan railtime:livekit-check
```

Erwartung: „Server-API erreichbar“. Damit sind Anrufe zwischen zwei Browsern bereits funktionsfähig (eingebautes LiveKit-TURN inklusive).

4. Phase 2 — coturn als Härtingsstufe (optional, empfohlen für strenge Firmennetze)

Das eingebaute LiveKit-TURN (Abschnitt 3) deckt die meisten Fälle ab. coturn lohnt sich, wenn TURN-Traffic über einen **separat kontrollierbaren Dienst mit statischen Zugangsdaten** laufen soll (so im ursprünglichen Fahrplan vorgesehen).

4.1 Zertifikat für coturn

coturn braucht (anders als Caddy) direkte Zertifikatsdateien:

```
apt-get -y install certbot
systemctl stop railtime-media    # Port 80 kurz freigeben
certbot certonly --standalone -d turn.rail-time.de
systemctl start railtime-media
```

Renewal-Hook `/etc/letsencrypt/renewal-hooks/deploy/restart-coturn.sh`:

```
#!/bin/sh
cd /opt/railtime-media && docker compose --profile coturn restart coturn
```

4.2 `/opt/railtime-media/turnserver.conf`

```
listening-port=3478
tls-listening-port=5349
realm=turn.rail-time.de
fingerprint

# Statische Zugangsdaten:
lt-cred-mech
user=railtime:<langes-statisches-passwort>

min-port=49160
max-port=49960
cert=/etc/letsencrypt/live/turn.rail-time.de/fullchain.pem
pkey=/etc/letsencrypt/live/turn.rail-time.de/privkey.pem

# Haertung: Relay in private Netze verbieten
no-multicast-peers
denied-peer-ip=10.0.0.0-10.255.255.255
denied-peer-ip=172.16.0.0-172.31.255.255
denied-peer-ip=192.168.0.0-192.168.255.255
```

> Bei coturn-Betrieb den `turn`-Block in `livekit.yaml` entfernen und den `> turn.rail-time.de:5349`-Block aus dem Caddyfile streichen (coturn > terminiert sein TLS selbst).

4.3 Starten und in der App umschalten

```
cd /opt/railtime-media && docker compose --profile coturn up -d
```

Laravel-`.env` (Plesk-Server):

```
LIVEKIT_TURN_MODE=coturn
TURN_URL=turns:turn.rail-time.de:5349
TURN_USERNAME=railtime
TURN_CREDENTIAL=<langes-statisches-passwort>
```

`php artisan config:clear` — der Token-Endpoint liefert die coturn-Daten ab sofort automatisch an alle Clients aus.

Test: `https://icetest.info` (oder Trickle-ICE-Seite) mit `turns:turn.rail-time.de:5349` + Zugangsdaten → es muss ein relay-Kandidat erscheinen.

5. Phase 8 — Test und Absicherung

5.1 Funktionsprüfung

- ☐ `php artisan railtime:livekit-check` grün (Scheduler-Eintrag empfohlen).
- ☐ Anruf zwischen zwei Browsern im selben Netz (Chat öffnen → Kamera-Symbol).
- ☐ Klingel-Overlay erscheint beim Gegenüber, Annehmen führt in den Anruf.
- ☐ Webhook-Kontrolle: Nach Anrufende steht der Raum in der DB auf `ended` (Tabelle `rooms`).

5.2 Netz-Matrix

Szenario	Erwartung
Büro-LAN (UDP frei)	Direkte UDP-Medien
Firmen-Firewall, UDP blockiert	Fallback TCP 7881
Nur 443/tcp erlaubt	TURN-TLS; im Client testweise erzwingen: <code>iceTransportPolicy: 'relay'</code>
Mobilfunk, WLAN↔LTE-Wechsel	LiveKit-Reconnect, Gespräch überlebt
Zug-WLAN (hoher Verlust)	Qualität degradiert, Anruf bricht nicht ab

5.3 Geräte

Chrome/Edge/Firefox Desktop, Safari macOS, **iOS-Safari als installierte PWA** (Web-Push-Klingeln erst ab iOS 16.4 und nur bei installierter App; das Reverb-Klingeln in offenen Tabs ist der Primärweg), Android-Chrome-PWA.

5.4 Last und Chaos

- Gruppenanruf mit 4 / 8 / 15 Teilnehmern; CPU/Bandbreite der VM beobachten (`docker stats`).
- `livekit-cli load-test` für synthetische Last.
- LiveKit-Container mitten im Anruf neu starten → Clients reconnecten, DB heilt über Webhooks.
- Reverb stoppen → laufender Anruf bleibt bestehen (Medien sind unabhängig).

5.5 Betrieb

- **Monitoring:** Uptime-Check auf `https://livekit.rail-time.de/` + `railtime:livekit-check` im Laravel-Scheduler; optional LiveKit-Prometheus (`prometheus_port: 6789`, nur intern freigeben).
- **Updates:** `docker compose pull && docker compose up -d` (Wartungsfenster, wirft laufende Anrufe).
- **Backups:** Die VM ist zustandslos — nur `/opt/railtime-media/` sichern. Anrufrdaten liegen in der App-Datenbank (bestehendes Backup).
- **DSGVO:** Bei externem VM-Hoster einen Auftragsverarbeitungsvertrag ergänzen.

Anhang — Plan B: alles auf dem Plesk-Server

Nur wenn keine zweite VM möglich ist: Docker Engine auf dem Plesk-Host, LiveKit im Host-Netz, TLS für `livekit.rail-time.de` über Plesk („Apache & nginx-Einstellungen“ → zusätzliche nginx-Direktiven → Proxy auf `127.0.0.1:7880`, gleiches Muster wie der Reverb-Proxy). Nachteile: kein TURN-TLS auf 443 (Plesk belegt den Port — die strengsten Firewalls bleiben außen vor), Plesk-Updates können Firewall-Regeln für die UDP-Range zurücksetzen, CPU-Konkurrenz zwischen PHP-FPM und der SFU.